

Universidad Mariano Galvez



Datos personales

9490-24-12144

Kerwin Israel Cuj Pernillo

Ingeniería de Sistemas

Programación 3

MANUAL TECNICO DE TOTITO

Sección A

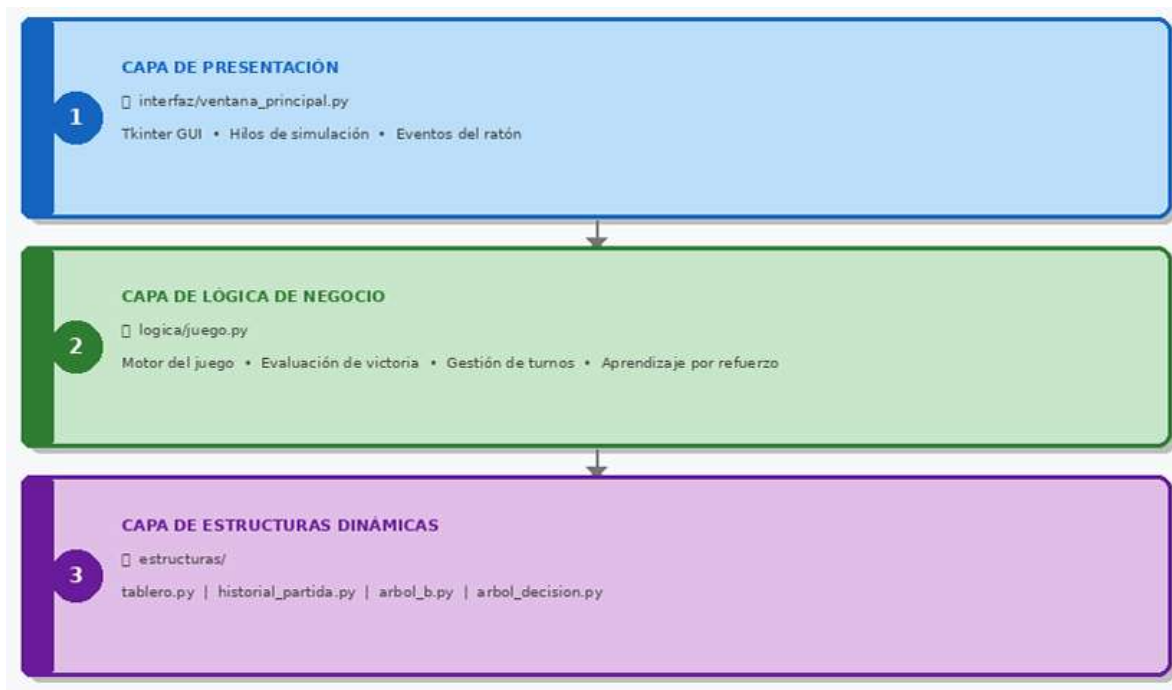
# MANUAL TECNICO

## 1. INTRODUCCIÓN Y ESQUEMA CONCEPTUAL

El presente proyecto implementa un agente de Inteligencia Artificial capaz de aprender a jugar el juego clásico "Totito" (3 \times 3) mediante un enfoque de aprendizaje por refuerzo. El sistema no depende de bases de datos externas ni de librerías de IA comerciales; toda la lógica de control, el almacenamiento de estados y el historial se gestionan a través de estructuras de datos lineales y no lineales de asignación dinámica de memoria creadas desde cero.

### Esquema Jerárquico del Flujo de Datos

El sistema se organiza en tres capas bien definidas que separan presentación, lógica de negocio y persistencia. Cada capa se comunica exclusivamente con la inmediatamente inferior, garantizando desacoplamiento y mantenimiento independiente:



## ARQUITECTURA DEL SISTEMA Y COMPONENTES

El software utiliza un patrón de diseño arquitectónico multicapa para garantizar el desacoplamiento de componentes:

**-Capa de Presentación (interfaz/ventana\_principal.py):** Controla el ciclo de vida de la interfaz gráfica basada en la librería estándar tkinter. Maneja hilos de simulación ligera y captura los eventos del puntero del ratón en las casillas.

**Capa de Lógica de Negocio (logica/juego.py):** Centraliza el motor del juego. Contiene el algoritmo que evalúa las condiciones de victoria, gestiona los turnos alternos y aplica las fórmulas matemáticas de castigo o recompensa al agente al finalizar las partidas.

**Capa de Estructuras Dinámicas (estructuras/):** Aloja los contenedores enlazados por punteros explicitados:

-tablero.py: Administra la grilla física del juego.

-historial\_partida.py: Almacena temporalmente los pasos secuenciales de la partida activa.

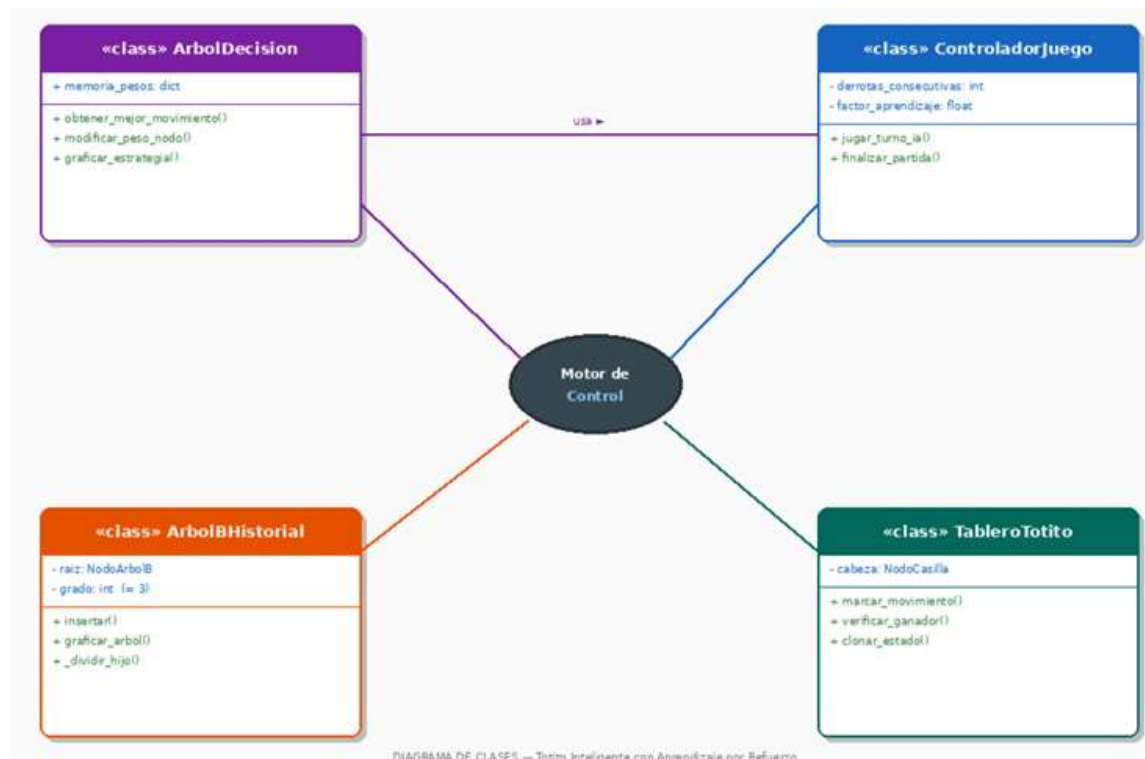
-arbol\_b.py: Mantiene el registro histórico ordenado.

-arbol\_decision.py: Contiene el modelo analítico de pesos que rige el comportamiento de la IA.

Toda estructura de datos fue implementada desde cero mediante nodos enlazados por punteros explicitados, sin utilizar colecciones nativas del sistema operativo. Esto garantiza control total sobre la gestión dinámica de memoria, cumpliendo los requisitos del curso de Programación III.

### Diagrama de clases y relaciones.

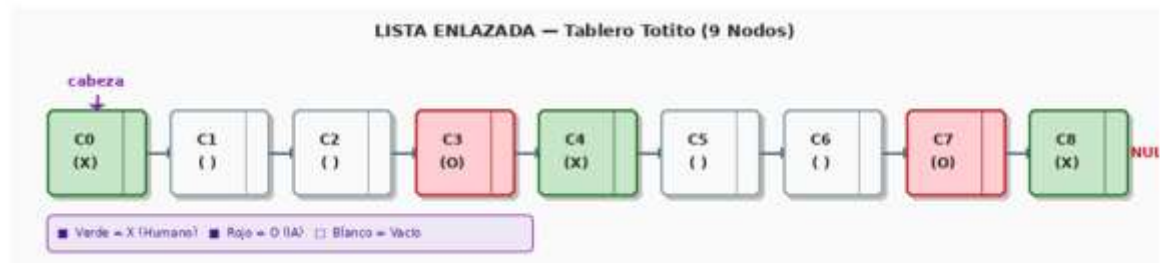
A continuación se presenta el mapa completo de componentes del código fuente, con sus atributos, métodos y relaciones de dependencia:



## ESPECIFICACIÓN DETALLADA DE CLASES

### Clase TableroTotito

- **Responsabilidad:** Controlar la integridad del tablero y el estado de ocupación de las celdas.



- **Atributos:** cabeza (Instancia que apunta al primer elemento de la secuencia de casillas).
- **Métodos Principales:**
  - marcar\_movimiento(posicion, simbolo): Valida que la celda se encuentre vacía e inserta el valor (X / O).
  - verificar\_ganador(): Realiza el análisis de las 8 combinaciones posibles de victoria sobre la estructura de nodos.
  - clonar\_estado(): Recorre los nodos y los compacta en un valor tipo cadena de texto (ej. X--O--X-O) para ser usado como identificador único.

### Clase ArbolDecision

- **Responsabilidad:** Funcionar como el núcleo cognitivo de la IA, ponderando la conveniencia de los movimientos según las experiencias acumuladas.
- **Atributos:** memoria\_pesos (Estructura de llaves anidadas que vincula un estado de tablero con el peso flotante de cada una de sus 9 casillas).
- **Métodos Principales:**
  - obtener\_mejor\_movimiento(tablero, derrotas): Elige el índice de la casilla que posee el valor de peso más alto. Incorpora subrutinas de prioridad posicional y el algoritmo defensivo de bloqueo.
  - graficar\_estrategia(id\_partida): Recorre los nodos del diccionario de conocimiento y exporta la jerarquía a sintaxis DOT para el renderizado de mapas visuales.

### Clase ArbolBHistorial

- **Responsabilidad:** Almacenar de manera indexada e inmutable los reportes de los juegos concluidos.



- **Atributos:** raíz (Nodo raíz del árbol), grado (Parámetro entero que define la capacidad máxima de claves por página, configurado en 3).
- **Métodos Principales:**
  - `insertar(id_partida, resumen, tablero_final)`: Inserta la clave en el nodo correspondiente. Si la página excede el límite del grado (M-1), invoca el método de división de páginas.
  - `_dividir_hijo(nodo_padre, i, nodo_hijo)`: Realiza la partición de un nodo lleno, promoviendo la clave mediana hacia el nivel superior para mantener el balance perfecto del árbol ( $O(\log n)$ ).

## 5. JUSTIFICACIÓN DE LAS ESTRUCTURAS DE DATOS

### Lista Simplemente Enlazada (Mapeo de Celdas y Pasos de Juego)

- **Propósito:** Modelar el tablero físico de juego y el histórico de movimientos inmediatos de la partida activa.
- **Justificación:** El uso de nodos enlazados mediante punteros directos evita el uso de colecciones estáticas indexadas de forma nativa por el sistema operativo, garantizando que el uso de memoria sea dinámico y estrictamente controlado por el programa de acuerdo con los requisitos del curso.

### Árbol de Decisión Estructurado (Diccionario de Pesos)

- **Propósito:** Representar los caminos lógicos que puede tomar una partida de Totito.
- **Justificación:** Cada estado del juego representa un nodo del cual se desprenden los hijos (las jugadas válidas). Esta aproximación no lineal permite búsquedas eficientes en tiempo de ejecución para que la respuesta de la IA sea inmediata.

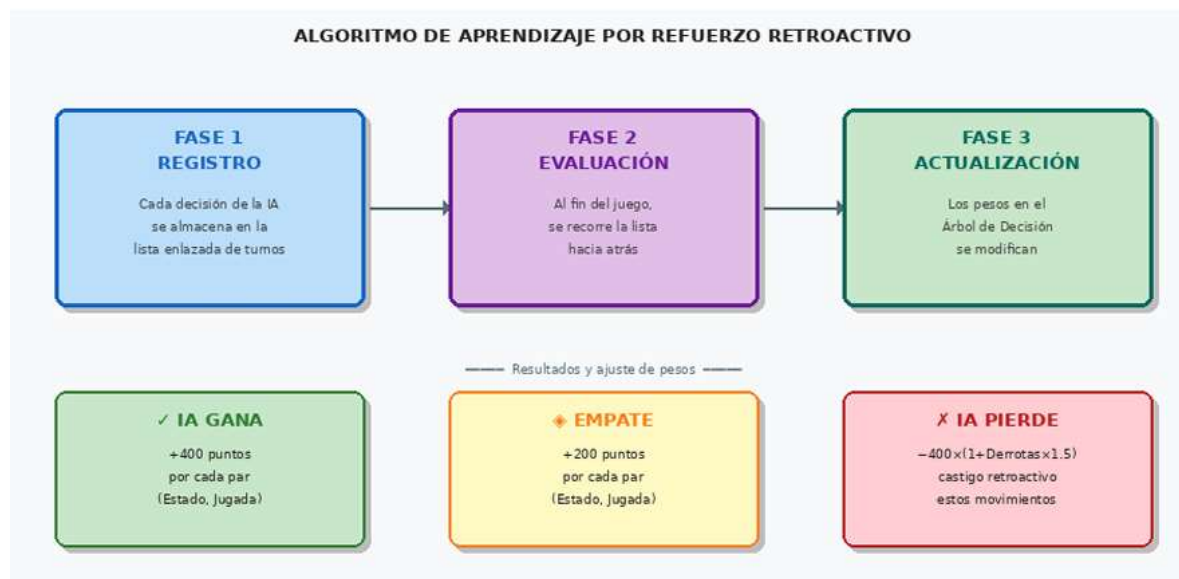
### Árbol B de Orden 3 (Indexación del Historial)

- **Propósito:** Almacenar de forma perenne la información de las partidas de la sesión de ejecución.
- **Justificación:** Un árbol binario común corre el riesgo de desbalancearse e incrementar su complejidad de búsqueda a  $O(n)$  si los datos ingresan ordenados correlativamente (1, 2, 3...). El Árbol B soluciona esto auto-balanceándose durante las inserciones, manteniendo búsquedas siempre logarítmicas ( $O(\log n)$ ) sin importar la cantidad de elementos almacenados.

| Estructura        | Operación                     | Complejidad   |
|-------------------|-------------------------------|---------------|
| Lista Enlazada    | Inserción / Acceso secuencial | $O(n)$        |
| Lista Enlazada    | Clonar estado del tablero     | $O(9) = O(1)$ |
| Árbol de Decisión | Consulta por estado           | $O(1)$ hash   |
| Árbol B Orden 3   | Búsqueda / Inserción          | $O(\log n)$   |
| Árbol B Orden 3   | División de nodo lleno        | $O(M)$        |

## 6. LÓGICA DEL APRENDIZAJE POR REFUERZO IMPLEMENTADO

El agente aprende de manera autónoma a través de un mecanismo de penalizaciones y recompensas basadas en un algoritmo de refuerzo retrógrado:



1. **Fase de Registro:** Durante el desarrollo del juego, cada decisión tomada por la IA se concatena en una lista enlazada.
2. **Fase de Evaluación:** Al declararse el fin del juego, el sistema procesa la lista hacia atrás:
  - **Si la IA Gana:** Cada par (Estado, Jugada) recibe una bonificación de **+400 puntos**, reforzando la probabilidad de repetir esos movimientos en el futuro.
  - **Si la IA Empata:** Recibe una bonificación moderada de **+200 puntos**.
  - **Si la IA Pierde:** El sistema aplica un castigo drástico. Se calcula un factor de aprendizaje escalado basado en la cantidad de derrotas consecutivas:

$\text{Puntos de Ajuste} = 400 \times (1.0 + \text{Derrotas Consecutivas} \times 1.5)$

Este puntaje se resta (valor negativo) de los nodos del camino que condujeron a la derrota, obligando a la IA a descartar esa línea de juego en futuras partidas.

## 7. FLUJO COMPLETO DE UNA PARTIDA

1. Se inicializan las estructuras en blanco y se despliega la interfaz gráfica.
2. El jugador humano selecciona una casilla vacía en la pantalla (X).
3. El controlador captura el cambio de estado y actualiza el tablero enlazado.
4. Se verifica que la jugada no sea ganadora para el humano.
5. **Turno de la IA:** Se extrae el string del tablero actual. Si existen dos derrotas consecutivas registradas en la sesión, la IA activa el "Modo Crítico" y realiza un escaneo lineal de bloqueo automático. Si no, busca en el Árbol de Decisión la casilla que posea el puntaje más alto para ese estado.
6. La IA efectúa su jugada (O) y registra el paso en la lista de turnos.
7. Al cumplirse la condición de fin de juego (tres en línea o tablero lleno), se detiene el reloj de la partida.
8. Se recorre la lista de movimientos para sumar o restar los pesos en el Árbol de Decisión.
9. Se empaquetan los datos del juego y se insertan en el Árbol B Historial.
10. Se invoca de forma obligatoria a la subrutina Graphviz para exportar el plano de conocimiento actual del sistema a un archivo físico .png.

